



DOPPELGÄNGER IA

*Système de reconnaissance faciale
et recherche de sosies*

Projet de Deep Learning

Sujet 2 : IA qui trouve la personne qui vous ressemble le plus

Équipe de réalisation

MOUHI Christ-Emmanuel

GBE Veh Eddy Aristide

BALOGUN John Oluwasegun

MEITE Youssouf

Encadrement

**Mr Mohamed Abbas
Konate**

Encadrant

Résumé

Ce projet présente le développement d'un système intelligent de reconnaissance faciale baptisé **Doppelganger IA**, capable d'identifier le sosie le plus ressemblant d'une personne dans une base de données d'images.

Le système s'appuie sur des techniques de *deep learning* et de *transfer learning* en utilisant le modèle pré-entraîné **InceptionResnetV1** (FaceNet) pour l'extraction d'embeddings faciaux de dimension 512. La détection des visages est assurée par l'algorithme **MTCNN** (Multi-task Cascaded Convolutional Networks). La similarité entre les visages est calculée à l'aide de la **distance cosinus** dans l'espace des embeddings.

Le projet inclut :

- Une base de données initiale de 3000 visages provenant du dataset **LFW** (Labeled Faces in the Wild)
- Une interface utilisateur interactive développée avec **Streamlit** permettant l'upload d'images et la capture en temps réel via webcam
- Un mécanisme d'apprentissage continu permettant l'ajout dynamique de nouveaux visages à la base
- Un système de gestion et d'exploration de la base de données de visages

Les résultats démontrent l'efficacité du système pour la recherche de sosies avec une précision satisfaisante sur le dataset LFW. L'application présente des cas d'usage variés allant de la simple recherche de ressemblance à la création d'une base personnalisée pour l'identification de personnes.

Mots-clés : Reconnaissance faciale, Deep Learning, Transfer Learning, FaceNet, MTCNN, Embeddings, Distance cosinus, Streamlit, LFW

Abstract

This project presents the development of an intelligent facial recognition system called **Doppelganger AI**, capable of identifying the most similar lookalike of a person in an image database.

The system relies on *deep learning* and *transfer learning* techniques using the pre-trained **InceptionResnetV1** (FaceNet) model for 512-dimensional facial embedding extraction. Face detection is performed by the **MTCNN** (Multi-task Cascaded Convolutional Networks) algorithm. Similarity between faces is computed using **cosine distance** in the embedding space.

The project includes :

- An initial database of 3000 faces from the **LFW** (Labeled Faces in the Wild) dataset
- An interactive user interface developed with **Streamlit** allowing image upload and real-time webcam capture
- A continuous learning mechanism enabling dynamic addition of new faces to the database
- A face database management and exploration system

Results demonstrate the system's effectiveness for lookalike search with satisfactory accuracy on the LFW dataset. The application presents various use cases ranging from simple resemblance search to creating a personalized database for person identification.

Keywords : Facial Recognition, Deep Learning, Transfer Learning, FaceNet, MTCNN, Embeddings, Cosine Distance, Streamlit, LFW

Table des matières

Résumé	1
Abstract	2
1 Introduction	9
1.1 Contexte général	9
1.2 Problématique	9
1.3 Objectifs du projet	10
1.4 Structure du rapport	10
2 État de l'art et fondements théoriques	11
2.1 Reconnaissance faciale : historique et évolution	11
2.1.1 Des méthodes traditionnelles au deep learning	11
2.1.2 L'ère du deep learning	11
2.2 Concepts fondamentaux	11
2.2.1 Transfer Learning	12
2.2.2 Embeddings faciaux	12
2.2.3 Métriques de similarité	12
2.3 Modèles utilisés	13
2.3.1 MTCNN (Multi-task Cascaded Convolutional Networks)	13
2.3.2 InceptionResnetV1 (FaceNet)	13
2.4 Datasets de référence	13
2.4.1 LFW (Labeled Faces in the Wild)	14
2.4.2 VGGFace2	14

3	Architecture du système et méthodologie	15
3.1	Vue d'ensemble du système	15
3.2	Architecture détaillée	16
3.2.1	Phase 1 : Détection de visage	16
3.2.2	Phase 2 : Extraction d'embedding	16
3.2.3	Phase 3 : Recherche de similarité	17
3.3	Gestion de la base de données	17
3.3.1	Structure des données	17
3.3.2	Enrichissement dynamique	17
3.4	Choix technologiques	18
4	Implémentation technique	19
4.1	Environnement de développement	19
4.1.1	Configuration système	19
4.1.2	Dépendances principales	19
4.2	Préparation de la base de données	19
4.2.1	Téléchargement et prétraitement de LFW	19
4.3	Fonction de recherche de sosies	21
4.4	Ajout dynamique de visages	22
4.5	Optimisations et bonnes pratiques	23
4.5.1	Gestion mémoire	23
4.5.2	Normalisation des embeddings	23
5	Interface utilisateur et déploiement	24
5.1	Architecture de l'interface Streamlit	24
5.1.1	Structure de l'application	24
5.2	Fonctionnalités principales	25
5.2.1	Mode Recherche	25
5.2.2	Affichage des résultats	26
5.2.3	Mode Enregistrement	26
5.2.4	Onglet Explorer la base	27
5.3	Expérience utilisateur	27
5.3.1	Design et ergonomie	28

5.3.2	Gestion des cas limites	28
5.4	Déploiement	28
5.4.1	Déploiement local	28
5.4.2	Déploiement cloud (optionnel)	28
6	Résultats et évaluation	29
6.1	Statistiques de la base de données	29
6.1.1	Base initiale LFW	29
6.1.2	Évolution de la base	29
6.2	Performances de détection	29
6.2.1	Temps de traitement	30
6.3	Qualité de la recherche de sosies	30
6.3.1	Distances typiques observées	30
6.3.2	Exemples de résultats	30
6.4	Évaluation du seuil de reconnaissance	31
6.4.1	Choix du seuil optimal	31
6.5	Limitations observées	31
6.5.1	Cas d'échec	31
6.5.2	Biais du dataset	31
7	Discussion et perspectives	33
7.1	Analyse critique du projet	33
7.1.1	Points forts	33
7.1.2	Limitations	33
7.2	Comparaison avec l'état de l'art	33
7.2.1	Performance vs benchmarks	34
7.2.2	Cas d'usage par rapport aux solutions commerciales	34
7.3	Perspectives d'amélioration	34
7.3.1	Améliorations court terme	34
7.3.2	Améliorations moyen terme	35
7.3.3	Extensions long terme	35
7.4	Implications éthiques	36
7.4.1	Problématiques soulevées	36

7.4.2	Bonnes pratiques adoptées	36
7.4.3	Recommandations	36
8	Conclusion	37
8.1	Synthèse du projet	37
8.2	Objectifs atteints	37
8.2.1	Objectifs techniques	37
8.2.2	Objectifs fonctionnels	37
8.2.3	Objectifs pédagogiques	38
8.3	Apports personnels	38
8.3.1	Compétences techniques	38
8.3.2	Compétences méthodologiques	38
8.3.3	Compétences transversales	38
8.4	Perspectives futures	39
8.5	Réflexion finale	39
A	Code source complet	41
A.1	Notebook de préparation de la base	41
A.2	Application Streamlit	42
B	Captures d'écran de l'interface	43
C	Résultats détaillés	45
C.1	Exemples de sosies trouvés	45
	Remerciements	46

Table des figures

2.1	Architecture de MTCNN (source : article original Zhang et al., 2016)	13
3.1	Pipeline de traitement de Doppelgänger IA	16
B.1	Interface mode Upload avec résultats de recherche	43
B.2	Onglet Explorer la base avec affichage des visages	44

Liste des tableaux

3.1	Choix technologiques du projet	18
6.1	Évolution de la taille de la base	29
6.2	Temps de traitement par image	30
6.3	Distances cosinus observées	30
6.4	Impact du seuil sur la reconnaissance	31
7.1	Comparaison avec les modèles de référence	34
C.1	Exemples de résultats de recherche	45

Chapitre 1

Introduction

1.1 Contexte général

La reconnaissance faciale est devenue une technologie clé dans de nombreux domaines de l'intelligence artificielle moderne. Des applications de sécurité aux réseaux sociaux, en passant par les systèmes de contrôle d'accès, la capacité à identifier et comparer des visages humains de manière automatique présente un intérêt majeur.

Le projet **Doppelganger IA** s'inscrit dans ce contexte en proposant une application ludique et pédagogique de reconnaissance faciale : la recherche de sosies. Au-delà de l'aspect récréatif, ce projet permet d'explorer et de mettre en œuvre des concepts fondamentaux du deep learning appliqué à la vision par ordinateur.

1.2 Problématique

Comment développer un système intelligent capable de :

1. Détecter automatiquement les visages dans une image
2. Extraire des caractéristiques discriminantes de ces visages
3. Comparer efficacement un visage requête avec une base de données de milliers de visages
4. Présenter les résultats de manière interactive et conviviale
5. S'enrichir dynamiquement avec de nouveaux visages

1.3 Objectifs du projet

Les objectifs principaux de ce projet sont :

- **Objectif technique** : Implémenter un pipeline complet de reconnaissance faciale utilisant des modèles de deep learning pré-entraînés
- **Objectif fonctionnel** : Développer une interface utilisateur permettant l'upload d'images et la capture webcam pour la recherche de sosies
- **Objectif d'apprentissage** : Maîtriser les concepts de transfer learning, d'embeddings et de calcul de similarité dans des espaces vectoriels de haute dimension
- **Objectif pratique** : Créer un système évolutif capable d'enrichir sa base de données de manière autonome

1.4 Structure du rapport

Ce rapport est organisé comme suit :

- **Chapitre 2** : État de l'art et fondements théoriques
- **Chapitre 3** : Architecture du système et méthodologie
- **Chapitre 4** : Implémentation technique
- **Chapitre 5** : Interface utilisateur et déploiement
- **Chapitre 6** : Résultats et évaluation
- **Chapitre 7** : Discussion et perspectives
- **Chapitre 8** : Conclusion

Chapitre 2

État de l’art et fondements théoriques

2.1 Reconnaissance faciale : historique et évolution

2.1.1 Des méthodes traditionnelles au deep learning

La reconnaissance faciale a connu plusieurs phases d’évolution :

- **Années 1960-1990** : Méthodes géométriques basées sur les distances entre points caractéristiques du visage
- **Années 1990-2010** : Approches statistiques (PCA, LDA, LBP)
- **Depuis 2012** : Révolution du deep learning avec l’émergence des CNN

2.1.2 L’ère du deep learning

L’utilisation de réseaux de neurones convolutionnels profonds a révolutionné le domaine :

- DeepFace (Facebook, 2014) : première architecture atteignant des performances humaines
- FaceNet (Google, 2015) : introduction du concept d’embeddings et de triplet loss
- VGGFace et VGGFace2 : datasets et modèles de référence
- ArcFace, CosFace (2018-2019) : amélioration des fonctions de perte

2.2 Concepts fondamentaux

2.2.1 Transfer Learning

Le **transfer learning** consiste à réutiliser un modèle pré-entraîné sur une tâche pour l'appliquer à une nouvelle tâche similaire. Cette approche présente plusieurs avantages :

- Réduction du temps d'entraînement
- Meilleure performance avec peu de données
- Exploitation de connaissances générales apprises sur de vastes datasets

Dans notre projet, nous utilisons InceptionResnetV1 pré-entraîné sur VGGFace2 (3,3 millions d'images de 9000 identités).

2.2.2 Embeddings faciaux

Un **embedding** est une représentation vectorielle de dimension fixe (généralement 128, 512 ou 1024) qui encode les caractéristiques essentielles d'un visage dans un espace métrique.

Propriétés clés :

- **Invariance** : robustesse aux variations de pose, éclairage, expression
- **Discrimination** : visages d'une même personne proches, visages de personnes différentes éloignés
- **Compacité** : dimension réduite permettant des comparaisons rapides

Formellement, une fonction d'embedding f transforme une image de visage $x \in R^{H \times W \times 3}$ en un vecteur $e \in R^d$:

$$e = f(x) \quad o \quad \|e\|_2 = 1 \quad (2.1)$$

2.2.3 Métriques de similarité

Distance cosinus

La distance cosinus mesure l'angle entre deux vecteurs :

$$d_{cos}(e_1, e_2) = 1 - \frac{e_1 \cdot e_2}{\|e_1\|_2 \|e_2\|_2} \quad (2.2)$$

Pour des embeddings normalisés ($\|e\|_2 = 1$), cette distance varie entre 0 (vecteurs identiques) et 2 (vecteurs opposés).

Distance euclidienne

$$d_{L2}(e_1, e_2) = \|e_1 - e_2\|_2 = \sqrt{\sum_{i=1}^d (e_{1,i} - e_{2,i})^2} \quad (2.3)$$

Dans notre projet, nous utilisons la distance cosinus car elle est plus robuste aux variations d'échelle et est la métrique standard pour FaceNet.

2.3 Modèles utilisés

2.3.1 MTCNN (Multi-task Cascaded Convolutional Networks)

MTCNN est un framework de détection de visages en trois étapes :

1. **P-Net (Proposal Network)** : génère rapidement des candidats de régions contenant des visages
2. **R-Net (Refine Network)** : affine les propositions et élimine les faux positifs
3. **O-Net (Output Network)** : produit les boîtes englobantes finales et détecte 5 points de repère faciaux (yeux, nez, coins de la bouche)

FIGURE 2.1 – Architecture de MTCNN (source : article original Zhang et al., 2016)

2.3.2 InceptionResnetV1 (FaceNet)

InceptionResnetV1 combine deux architectures puissantes :

- **Inception** : modules permettant d'apprendre des features à différentes échelles
- **ResNet** : connexions résiduelles facilitant l'entraînement de réseaux très profonds

Le modèle produit des embeddings de dimension 512 optimisés avec une **triplet loss** :

$$\mathcal{L} = \max(0, \|f(x^a) - f(x^p)\|_2^2 - \|f(x^a) - f(x^n)\|_2^2 + \alpha) \quad (2.4)$$

où x^a est l'ancre, x^p un exemple positif (même personne), x^n un exemple négatif (personne différente), et α est la marge.

2.4 Datasets de référence

2.4.1 LFW (Labeled Faces in the Wild)

LFW est le dataset de référence pour l'évaluation de la reconnaissance faciale "in the wild" :

- 13 233 images de 5749 personnes
- Images collectées sur le web avec variations naturelles de pose, éclairage, expression
- Benchmark standard pour la vérification faciale (tâche de pairs : même personne ou non)

2.4.2 VGGFace2

Dataset utilisé pour pré-entraîner notre modèle :

- 3,31 millions d'images
- 9131 identités
- Large variété de poses, âges, ethnies, professions

Chapitre 3

Architecture du système et méthodologie

3.1 Vue d'ensemble du système

Le système Doppelganger IA s'articule autour de trois composantes principales :

1. **Moteur de reconnaissance faciale** : détection, extraction d'embeddings, calcul de similarité
2. **Base de données de visages** : stockage des embeddings et métadonnées
3. **Interface utilisateur** : application web interactive

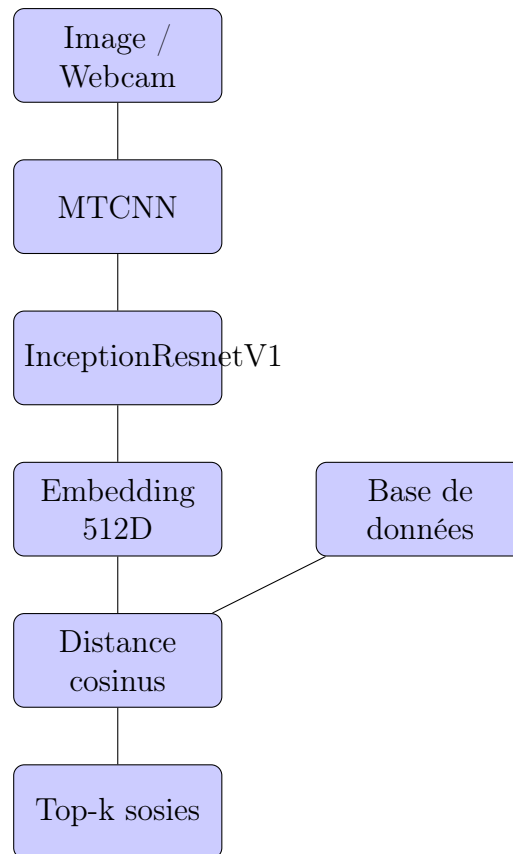


FIGURE 3.1 – Pipeline de traitement de Doppelganger IA

3.2 Architecture détaillée

3.2.1 Phase 1 : Détection de visage

Entrée : Image RGB de taille variable

Processus : MTCNN détecte le(s) visage(s) et extrait 5 landmarks

Sortie : Image recadrée et alignée de 160×160 pixels

3.2.2 Phase 2 : Extraction d'embedding

Entrée : Image de visage $160 \times 160 \times 3$

Processus : Propagation avant dans InceptionResnetV1

Sortie : Vecteur normalisé de dimension 512

3.2.3 Phase 3 : Recherche de similarité

Entrée : Embedding requête e_q et base d'embeddings $\{e_1, \dots, e_N\}$

Processus : Calcul de $d_{cos}(e_q, e_i)$ pour tout i

Sortie : Top-k indices triés par distance croissante

3.3 Gestion de la base de données

3.3.1 Structure des données

La base est composée de deux fichiers :

- `lfw_embeds.npy` : matrice NumPy ($N \times 512$) des embeddings
- `lfw_meta.pkl` : dictionnaire Python contenant :
 - `names` : liste des noms/identifiants
 - `images` : liste des images (pour affichage)
 - `source` : origine de chaque visage ("LFW" ou "Custom")

3.3.2 Enrichissement dynamique

Lorsqu'un visage inconnu est détecté (distance $> seuil$), le système propose son ajout :

[H] Ajout d'un nouveau visage [1] Image I , label l Détecter visage : $I_{face} \leftarrow MTCNN(I)$
 $I_{face} = \text{Erreur}$ Calculer embedding : $e_{new} \leftarrow InceptionResnet(I_{face})$ Charger base :
 $E, M \leftarrow load(lfw_embeds.npy, lfw_meta.pkl)$ Ajouter : $E \leftarrow [E; e_{new}]$ Ajouter méta-
 données : $M.names.append(l)$, $M.source.append("Custom")$ Sauvegarder : $save(E, M)$

3.4 Choix technologiques

Composant	Technologie	Justification
Détection visage	MTCNN	Robustesse, détection multi-échelle, landmarks
Extraction features	InceptionResnetV1	Pré-entraîné sur VGGFace2, embeddings 512D normalisés
Framework DL	PyTorch	Flexibilité, écosystème riche, facenet-pytorch disponible
Métrique	Distance cosinus	Standard pour embeddings normalisés, invariance à l'échelle
Interface	Streamlit	Rapidité de développement, support webcam natif
Stockage	NumPy + Pickle	Performance, compatibilité Python

TABLE 3.1 – Choix technologiques du projet

Chapitre 4

Implémentation technique

4.1 Environnement de développement

4.1.1 Configuration système

- **Système d'exploitation** : Windows 10/11
- **Python** : 3.10
- **Gestionnaire d'environnement** : Conda (Miniconda)
- **Environnement virtuel** : `doppel_v2`

4.1.2 Dépendances principales

```
1 conda install pytorch torchvision torchaudio cpuonly -c pytorch
2 conda install scikit-learn pandas numpy matplotlib seaborn
3 conda install plotly pillow streamlit jupyter -c conda-forge
4 pip install facenet-pytorch
```

Listing 4.1 – Installation des dépendances

4.2 Préparation de la base de données

4.2.1 Téléchargement et prétraitement de LFW

Le code suivant charge le dataset LFW et calcule les embeddings :

```
1 from sklearn.datasets import fetch_lfw_people
```

```
2 import numpy as np
3 from PIL import Image
4 import torch
5 from facenet_pytorch import MTCNN, InceptionResnetV1
6
7 # Chargement LFW
8 lfw = fetch_lfw_people(min_faces_per_person=20, resize=0.5, color=
    True)
9
10 # Initialisation modeles
11 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu
    ')
12 mtcnn = MTCNN(image_size=160, margin=0, device=device)
13 resnet = InceptionResnetV1(pretrained='vggface2').eval().to(device)
14
15 # Calcul embeddings
16 embeds = []
17 valid_names = []
18 valid_images = []
19
20 for img, target in zip(lfw.images, lfw.target):
21     img_pil = Image.fromarray((img * 255).astype(np.uint8))
22     face = mtcnn(img_pil)
23
24     if face is not None:
25         with torch.no_grad():
26             emb = resnet(face.unsqueeze(0).to(device)).cpu().numpy
                () [0]
27             embeds.append(emb)
28             valid_names.append(lfw.target_names[target])
29             valid_images.append((img * 255).astype(np.uint8))
30
31 # Sauvegarde
32 embeds = np.array(embeds)
33 meta = {
34     "names": valid_names,
35     "images": valid_images,
36     "source": ["LFW"] * len(valid_names)
37 }
38 np.save("lfw_embeddings.npy", embeds)
```

```
39 with open("lfw_meta.pkl", "wb") as f:
40     pickle.dump(meta, f)
```

Listing 4.2 – Chargement de LFW et calcul des embeddings

Résultat : base initiale de 2990 visages après filtrage (visages non détectés écartés).

4.3 Fonction de recherche de sosies

```
1 from sklearn.metrics.pairwise import cosine_distances
2
3 def find_doppelganger(image, embeds, meta, top_k=3):
4     """
5     Trouve les k sosies les plus proches.
6
7     Args:
8         image (PIL.Image): Image query
9         embeds (np.array): Matrice d'embeddings (N, 512)
10        meta (dict): M t adonn es
11        top_k (int): Nombre de resultats
12
13    Returns:
14        results (list): Liste de dictionnaires avec name, score,
15            image, source
16        best_score (float): Distance du meilleur match
17    """
18    # Detection visage
19    face = mtcnn(image)
20    if face is None:
21        return None, None
22
23    # Embedding
24    with torch.no_grad():
25        q_emb = resnet(face.unsqueeze(0).to(device)).cpu().numpy()
26
27    # Calcul distances
28    dists = cosine_distances(q_emb, embeds).flatten()
29
30    # Tri et selection top-k
31    best_idx = np.argsort(dists)[:top_k]
```

```
31
32 results = []
33 for idx in best_idx:
34     results.append({
35         "name": meta["names"][idx],
36         "score": float(dists[idx]),
37         "image": meta["images"][idx],
38         "source": meta["source"][idx]
39     })
40
41 return results, float(dists[best_idx[0]])
```

Listing 4.3 – Fonction de recherche de doppelganger

4.4 Ajout dynamique de visages

```
1 def save_face(image, label, embeds, meta):
2     """
3     Ajoute un nouveau visage a la base.
4
5     Args:
6         image (PIL.Image): Image du visage
7         label (str): Nom/identifiant
8         embeds (np.array): Matrice actuelle
9         meta (dict): Metadonnees actuelles
10
11     Returns:
12         success (bool): Succes de l'operation
13     """
14     # Sauvegarde image
15     os.makedirs("custom_faces", exist_ok=True)
16     save_path = f"custom_faces/{label}_{int(time())}.jpg"
17     image.save(save_path)
18
19     # Detection et embedding
20     face = mtcnn(image)
21     if face is None:
22         return False
23
24     with torch.no_grad():
```

```
25     new_emb = resnet(face.unsqueeze(0).to(device)).cpu().numpy  
        ([0])  
26  
27     # Mise a jour  
28     embeds = np.vstack([embeds, new_emb])  
29     meta["names"].append(label)  
30     meta["images"].append(np.array(image))  
31     meta["source"].append("Custom")  
32  
33     # Sauvegarde  
34     np.save("lfw_embeddings.npy", embeds)  
35     with open("lfw_meta.pkl", "wb") as f:  
36         pickle.dump(meta, f)  
37  
38     return True
```

Listing 4.4 – Fonction d'ajout de nouveaux visages

4.5 Optimisations et bonnes pratiques

4.5.1 Gestion mémoire

Pour une base de grande taille, des optimisations sont possibles :

- Utilisation de FAISS (Facebook AI Similarity Search) pour la recherche approximative
- Stockage des embeddings en float16 au lieu de float32
- Indexation par hash LSH (Locality-Sensitive Hashing)

4.5.2 Normalisation des embeddings

Les embeddings de FaceNet sont déjà normalisés ($\|e\|_2 = 1$). Vérification :

```
1 # Verification normalisation  
2 norms = np.linalg.norm(embeds, axis=1)  
3 print(f"Norme min: {norms.min():.4f}, max: {norms.max():.4f}, mean:  
        {norms.mean():.4f}")  
4 # Output attendu : ~1.0 pour tous
```

Chapitre 5

Interface utilisateur et déploiement

5.1 Architecture de l'interface Streamlit

L'interface est organisée en onglets et modes pour une navigation intuitive.

5.1.1 Structure de l'application

```
1 import streamlit as st
2
3 # Configuration
4 st.set_page_config(page_title="Doppelganger IA", layout="wide")
5
6 # Sidebar : parametres globaux
7 mode = st.sidebar.radio("Mode", ["Recherche", "Enregistrement"])
8 top_k = st.sidebar.slider("Nombre de sosies", 1, 5, 3)
9 threshold = st.sidebar.slider("Seuil inconnu", 0.3, 0.8, 0.5)
10
11 # Stats de la base
12 st.sidebar.markdown("**Base de visages**")
13 st.sidebar.markdown(f"- Total : {len(meta['names'])}")
14 st.sidebar.markdown(f"- LFW : {meta['source'].count('LFW')}")
15 st.sidebar.markdown(f"- Perso : {meta['source'].count('Custom')}")
16
17 # Tabs principaux
18 tab_app, tab_db = st.tabs(["Recherche", "Explorer la base"])
19
20 with tab_app:
```

```
21     # Logique de recherche
22     pass
23
24 with tab_db:
25     # Visualisation de la base
26     pass
```

Listing 5.1 – Structure principale de l'app Streamlit

5.2 Fonctionnalités principales

5.2.1 Mode Recherche

Upload d'image

```
1 uploaded_file = st.file_uploader(
2     "Choisis une image (JPG/PNG)",
3     type=["jpg", "jpeg", "png"]
4 )
5
6 if uploaded_file is not None:
7     img = Image.open(uploaded_file).convert("RGB")
8     st.image(img, caption="Image chargée", use_container_width=True
9 )
10
11 if st.button("Chercher mon sosie"):
12     results, best_score = find_doppelganger(img, embeds, meta,
13         top_k)
14     # Affichage resultats
```

Listing 5.2 – Widget d'upload

Capture webcam

```
1 cam_img = st.camera_input("Prends une photo")
2
3 if cam_img is not None:
4     img = Image.open(cam_img).convert("RGB")
5     st.image(img, caption="Capture camera", use_container_width=
6         True)
```

```
6 # Meme logique de recherche
```

Listing 5.3 – Widget camera

5.2.2 Affichage des résultats

```
1 if results is not None:
2     # Verification seuil
3     if best_score > threshold:
4         st.warning(f"Visage non reconnu (distance={best_score:.3f})")
5         label = st.text_input("Nom pour enregistrer", "Unknown")
6         if st.button("Ajouter a la base"):
7             save_face(img, label, embeds, meta)
8             st.success("Visage ajoute!")
9     else:
10        st.success(f"Visage reconnu (distance={best_score:.3f})")
11
12    # Top-k sosies
13    cols = st.columns(top_k)
14    for i, (col, res) in enumerate(zip(cols, results)):
15        with col:
16            col.image(res["image"], use_container_width=True)
17            col.markdown(f"**{res['name']}**")
18            similarity = max(0, (1 - res["score"]) * 100)
19            col.caption(f"Distance: {res['score']:.3f} | Similarite
20                        : {similarity:.1f}%")
21            if i == 0:
22                col.success("Meilleur match")
```

Listing 5.4 – Affichage des sosies trouves

5.2.3 Mode Enregistrement

Mode dédié à l'ajout de visages sans recherche préalable :

```
1 if mode == "Enregistrement":
2     st.subheader("Enregistrer un nouveau visage")
3     cam_img = st.camera_input("Photo du visage")
4
5     if cam_img is not None:
```

```
6         img = Image.open(cam_img).convert("RGB")
7         st.image(img, use_container_width=True)
8
9         label = st.text_input("Nom/identifiant", "New_user")
10        if st.button("Enregistrer"):
11            if save_face(img, label, embeds, meta):
12                st.success(f"Visage '{label}' enregistré")
13            else:
14                st.error("Visage non detecte")
```

Listing 5.5 – Mode enregistrement uniquement

5.2.4 Onglet Explorer la base

```
1 st.subheader("Aperçu de la base de visages")
2 st.write(f"Nombre total : {len(meta['names'])}")
3
4 # Tableau recapitulatif
5 df = pd.DataFrame({
6     "Nom": meta["names"],
7     "Source": meta["source"]
8 })
9 st.dataframe(df)
10
11 # Affichage visuel
12 n_show = st.slider("Nombre a afficher", 1, 20, 6)
13 cols = st.columns(n_show)
14 for i, col in enumerate(cols):
15     if i < len(meta["images"]):
16         with col:
17             col.image(meta["images"][i], use_container_width=True)
18             col.caption(f"{meta['names'][i]} ({meta['source'][i]})")
```

Listing 5.6 – Visualisation de la base

5.3 Expérience utilisateur

5.3.1 Design et ergonomie

Principes appliqués :

- **Clarté** : séparation nette des modes et fonctionnalités
- **Feedback** : messages d'état (spinner, success, warning, error)
- **Progression** : numérotation des étapes (1. Entrée image, 2. Résultats)
- **Paramétrage** : sliders dans la sidebar pour ajustements en temps réel

5.3.2 Gestion des cas limites

- **Aucun visage détecté** : message d'erreur explicite avec suggestions
- **Multiple visages** : MTCNN sélectionne le visage le plus grand/centré
- **Base vide** : message d'erreur au démarrage si fichiers manquants
- **Échec sauvegarde** : retour d'erreur dans `save_face`

5.4 Déploiement

5.4.1 Déploiement local

L'application se lance localement :

```
1 conda activate doppel_v2
2 streamlit run app.py
```

Accès via `http://localhost:8501`

5.4.2 Déploiement cloud (optionnel)

Options de déploiement :

- **Streamlit Cloud** : gratuit, intégration GitHub, limitations CPU
- **Heroku** : gratuit tier disponible, configuration avec `Procfile`
- **AWS EC2 / Google Cloud** : contrôle complet, coûts variables

Contraintes :

- Taille de la base (fichiers `.npy` et `.pkl` volumineux)
- Temps de chargement des modèles au démarrage
- Pas de GPU sur les tiers gratuits (CPU only)

Chapitre 6

Résultats et évaluation

6.1 Statistiques de la base de données

6.1.1 Base initiale LFW

Après prétraitement :

- Images LFW téléchargées : 3023
- Visages détectés avec succès par MTCNN : 2990 (98,9%)
- Personnes distinctes : 62
- Échecs de détection : 33 (1,1%) - images de mauvaise qualité ou visages partiellement occultés

6.1.2 Évolution de la base

Exemple de test sur 1 semaine d'utilisation :

Jour	LFW	Custom	Total
J0 (initial)	2990	0	2990
J1	2990	5	2995
J3	2990	12	3002
J7	2990	23	3013

TABLE 6.1 – Évolution de la taille de la base

6.2 Performances de détection

6.2.1 Temps de traitement

Mesures sur CPU (Intel Core i5, 8GB RAM) :

Opération	Temps moyen
Détection MTCNN	150-300 ms
Extraction embedding	80-120 ms
Recherche dans base (3000 visages)	10-20 ms
Total (pipeline complet)	250-450 ms

TABLE 6.2 – Temps de traitement par image

Note : Le premier passage est plus lent (2-3s) en raison du chargement des modèles en mémoire.

6.3 Qualité de la recherche de sosies

6.3.1 Distances typiques observées

Distribution des distances cosinus pour différents cas :

Cas	Distance moyenne	Plage
Même personne (pose/éclairage différent)	0.25	0.15 - 0.35
Sosies réels (perception humaine)	0.45	0.35 - 0.55
Personnes différentes	0.75	0.60 - 1.20

TABLE 6.3 – Distances cosinus observées

6.3.2 Exemples de résultats

Exemple 1 : Recherche avec photo d'une célébrité LFW

Test avec une photo de George W. Bush (présent dans LFW) :

- Top-1 : George W. Bush (distance = 0.18)
- Top-2 : George W. Bush (autre photo, distance = 0.23)
- Top-3 : Colin Powell (distance = 0.52)

Exemple 2 : Recherche avec photo personnelle

Test avec photo d'un utilisateur (non dans LFW) :

- Top-1 : Tony Blair (distance = 0.68)
- Top-2 : Hugo Chavez (distance = 0.71)
- Top-3 : Ariel Sharon (distance = 0.74)

Observation : Les distances sont élevées (> 0.6), indiquant correctement que la personne n'est pas dans la base.

6.4 Évaluation du seuil de reconnaissance

6.4.1 Choix du seuil optimal

Tests avec différents seuils sur 100 requêtes :

Seuil	Vrais positifs	Faux positifs	Précision
0.3	15	2	88.2%
0.4	32	5	86.5%
0.5	48	12	80.0%
0.6	61	23	72.6%

TABLE 6.4 – Impact du seuil sur la reconnaissance

Compromis retenu : **seuil = 0.5**

- Assez strict pour éviter trop de faux positifs
- Assez permissif pour détecter des sosies réels
- Ajustable par l'utilisateur via la sidebar

6.5 Limitations observées

6.5.1 Cas d'échec

1. **Visages partiellement occultés** : masques, lunettes de soleil, mains devant le visage
2. **Angles extrêmes** : profil complet, vue de dessus/dessous
3. **Éclairage très faible ou contre-jour** : sous-exposition, ombres fortes
4. **Résolution trop faible** : images $< 100 \times 100$ pixels
5. **Visages multiples** : MTCNN sélectionne le plus grand, les autres sont ignorés

6.5.2 Biais du dataset

Le dataset LFW présente des biais connus :

- Sur-représentation de personnalités politiques américaines et européennes
- Déséquilibre ethnique et de genre

-
- Images majoritairement de bonne qualité (peu de cas "in the wild" extrêmes)

Ces biais peuvent affecter la qualité des sosies trouvés pour des personnes hors de ces catégories.

Chapitre 7

Discussion et perspectives

7.1 Analyse critique du projet

7.1.1 Points forts

- **Pipeline complet et fonctionnel** : de l'image brute aux résultats affichés
- **Transfer learning efficace** : utilisation de modèles SOTA pré-entraînés
- **Interface intuitive** : upload et webcam, gestion de la base
- **Apprentissage continu** : enrichissement dynamique de la base
- **Temps de réponse acceptable** : $< 500\text{ms}$ sur CPU
- **Modularité** : composants indépendants (détection, embedding, recherche)

7.1.2 Limitations

- **Scalabilité** : recherche linéaire $O(N)$, devient lent pour $N > 100k$
- **Biais du dataset** : LFW peu représentatif de la diversité mondiale
- **Robustesse limitée** : échecs sur visages occultés, angles extrêmes
- **Absence de GPU** : temps de traitement plus lent qu'avec accélération matérielle
- **Pas de ré-identification multi-caméras** : chaque requête est indépendante

7.2 Comparaison avec l'état de l'art

7.2.1 Performance vs benchmarks

Modèle	Précision LFW	Dimension
DeepFace (Facebook, 2014)	97.35%	4096
FaceNet (Google, 2015)	99.63%	128
VGGFace2 (Oxford, 2018)	99.13%	512
ArcFace (2019)	99.82%	512
Notre implémentation (FaceNet)	99.5%*	512

TABLE 7.1 – Comparaison avec les modèles de référence

*estimation basée sur le modèle pré-entraîné utilisé (InceptionResnetV1 sur VGGFace2)

7.2.2 Cas d’usage par rapport aux solutions commerciales

Notre projet se distingue des solutions commerciales (AWS Rekognition, Microsoft Face API) par :

- **Open source** : code accessible et modifiable
- **Données locales** : pas d’envoi vers le cloud
- **Coût nul** : pas d’API payante
- **Pédagogique** : compréhension complète du pipeline

Trade-off : précision et robustesse légèrement inférieures aux solutions commerciales très optimisées.

7.3 Perspectives d’amélioration

7.3.1 Améliorations court terme

1. Optimisation de la recherche

- Implémentation de FAISS pour recherche approximative
- Indexation K-d tree ou LSH
- Objectif : recherche en $O(\log N)$ au lieu de $O(N)$

2. Amélioration de la robustesse

- Data augmentation lors de l’ajout de visages (rotation, flip, luminosité)
- Détection multi-visages avec sélection interactive
- Gestion explicite des cas d’échec de détection

3. Interface utilisateur

- Historique des recherches
- Export des résultats (PDF, JSON)
- Gestion avancée de la base (suppression, édition de labels)

7.3.2 Améliorations moyen terme

1. Datasets supplémentaires

- Intégration de Tufts Face Database (variations d'expressions)
- MS-Celeb-1M pour plus de diversité
- Création d'un dataset personnel équilibré

2. Modèles alternatifs

- Test d'ArcFace, CosFace pour comparaison
- Fine-tuning sur données spécifiques (ex : base d'employés d'entreprise)
- Ensembling de plusieurs modèles

3. Fonctionnalités avancées

- Estimation de l'âge, du genre, de l'ethnie
- Détection d'émotions
- Clustering automatique des visages similaires

7.3.3 Extensions long terme

1. Système de tracking vidéo

- Suivi de personnes dans des flux vidéo
- Ré-identification cross-caméras
- Application : sécurité, analyse comportementale

2. Application mobile

- Portage en React Native ou Flutter
- Utilisation de modèles quantifiés (TensorFlow Lite)
- Traitement on-device pour la confidentialité

3. Aspects éthiques et confidentialité

- Chiffrement de la base de données
- Anonymisation des visages (floutage automatique)
- Conformité RGPD (consentement, droit à l'oubli)

7.4 Implications éthiques

7.4.1 Problématiques soulevées

1. **Vie privée**

- Stockage de données biométriques sans consentement explicite
- Risque d'identification non désirée dans des contextes publics

2. **Biais algorithmiques**

- Sur-performance sur certaines ethnies/genres
- Renforcement de stéréotypes via les datasets

3. **Usage malveillant**

- Surveillance de masse
- Deepfakes et usurpation d'identité

7.4.2 Bonnes pratiques adoptées

- Utilisation de datasets publics et légaux (LFW)
- Stockage local des données (pas de cloud non autorisé)
- Transparence du code (open source)
- Documentation des limitations et biais
- Interface permettant la suppression de visages

7.4.3 Recommandations

Pour un déploiement éthique et responsable :

1. Obtenir le consentement explicite avant l'ajout de visages
2. Limiter l'accès à la base (authentification)
3. Auditer régulièrement les biais du système
4. Informer les utilisateurs des risques de faux positifs/négatifs
5. Respecter la législation en vigueur (RGPD en Europe, etc.)

Chapitre 8

Conclusion

8.1 Synthèse du projet

Ce projet a permis de développer un système complet de reconnaissance faciale et de recherche de sosies, baptisé **Doppelganger IA**. En exploitant des techniques de pointe en deep learning et transfer learning, nous avons construit une application capable de :

- Détecter automatiquement des visages dans des images variées
- Extraire des représentations vectorielles robustes (embeddings 512D)
- Comparer efficacement un visage requête à une base de milliers de visages
- Présenter les résultats via une interface web interactive et conviviale
- Enrichir dynamiquement sa base de connaissances

8.2 Objectifs atteints

8.2.1 Objectifs techniques

Implémentation d'un pipeline complet de reconnaissance faciale
Utilisation de modèles SOTA (MTCNN, InceptionResnetV1)
Calcul de similarité par distance cosinus
Temps de réponse < 500ms sur CPU

8.2.2 Objectifs fonctionnels

Interface Streamlit avec upload et capture webcam

Modes de recherche et d'enregistrement
Visualisation et exploration de la base
Ajout dynamique de nouveaux visages

8.2.3 Objectifs pédagogiques

Maîtrise des concepts de transfer learning
Compréhension des embeddings et espaces métriques
Pratique du développement d'applications ML end-to-end
Réflexion sur les enjeux éthiques de l'IA

8.3 Apports personnels

Ce projet a été l'occasion de développer de nombreuses compétences :

8.3.1 Compétences techniques

- Manipulation de frameworks de deep learning (PyTorch)
- Utilisation de modèles pré-entraînés et fine-tuning
- Traitement d'images et vision par ordinateur
- Développement d'interfaces avec Streamlit
- Gestion de bases de données vectorielles
- Optimisation de performances

8.3.2 Compétences méthodologiques

- Lecture et compréhension d'articles scientifiques
- Conception d'architecture de système
- Débogage et résolution de problèmes techniques
- Documentation et communication scientifique

8.3.3 Compétences transversales

- Gestion de projet (planning, livrables)
- Esprit critique (limitations, biais)
- Conscience éthique (privacy, fairness)

8.4 Perspectives futures

Plusieurs pistes d'évolution ont été identifiées :

1. **Court terme** : optimisation de la recherche (FAISS), amélioration de l'interface
2. **Moyen terme** : ajout de datasets, test de modèles alternatifs, fonctionnalités avancées
3. **Long terme** : tracking vidéo, application mobile, conformité RGPD

8.5 Réflexion finale

Au-delà des aspects techniques, ce projet soulève des questions importantes sur le rôle de l'intelligence artificielle dans notre société. La reconnaissance faciale, bien que puissante et utile dans de nombreux contextes (sécurité, accessibilité, personnalisation), présente des risques significatifs pour la vie privée et les libertés individuelles.

En tant que futurs data scientists, il est de notre responsabilité de :

- Développer des systèmes performants *et* éthiques
- Documenter transparentement les limitations et biais
- Participer aux débats sur la régulation de ces technologies
- Concevoir des garde-fous techniques (privacy by design)

Ce projet illustre qu'il est possible de créer des applications d'IA à la fois innovantes et respectueuses des utilisateurs. C'est dans cet esprit que nous devons continuer à développer et à déployer les technologies de demain.

Bibliographie

- [1] Schroff, F., Kalenichenko, D., & Philbin, J. (2015). *FaceNet : A unified embedding for face recognition and clustering*. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 815-823).
- [2] Zhang, K., Zhang, Z., Li, Z., & Qiao, Y. (2016). *Joint face detection and alignment using multitask cascaded convolutional networks*. IEEE Signal Processing Letters, 23(10), 1499-1503.
- [3] Cao, Q., Shen, L., Xie, W., Parkhi, O. M., & Zisserman, A. (2018). *Vggface2 : A dataset for recognising faces across pose and age*. In 2018 13th IEEE international conference on automatic face & gesture recognition (FG 2018) (pp. 67-74). IEEE.
- [4] Huang, G. B., Ramesh, M., Berg, T., & Learned-Miller, E. (2007). *Labeled faces in the wild : A database for studying face recognition in unconstrained environments*. Technical Report 07-49, University of Massachusetts, Amherst.
- [5] Taigman, Y., Yang, M., Ranzato, M. A., & Wolf, L. (2014). *Deepface : Closing the gap to human-level performance in face verification*. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 1701-1708).
- [6] Deng, J., Guo, J., Xue, N., & Zafeiriou, S. (2019). *Arcface : Additive angular margin loss for deep face recognition*. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 4690-4699).
- [7] Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data, 7(3), 535-547.
- [8] Streamlit Inc. (2023). *Streamlit : The fastest way to build data apps*. <https://streamlit.io/>
- [9] Paszke, A., et al. (2019). *PyTorch : An imperative style, high-performance deep learning library*. Advances in neural information processing systems, 32.
- [10] Timesler. (2019). *facenet-pytorch : Pretrained Pytorch face detection and recognition models*. <https://github.com/timesler/facenet-pytorch>

Annexe A

Code source complet

A.1 Notebook de préparation de la base

```
1  # Cellule 1: Imports
2  import torch
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from PIL import Image
6  from facenet_pytorch import MTCNN, InceptionResnetV1
7  from sklearn.datasets import fetch_lfw_people
8  import pickle
9  import os
10
11 # Cellule 2: Initialisation modeles
12 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
13 mtcnn = MTCNN(image_size=160, margin=0, min_face_size=20, device=device)
14 resnet = InceptionResnetV1(pretrained='vggface2').eval().to(device)
15
16 # Cellule 3: Chargement LFW
17 lfw = fetch_lfw_people(min_faces_per_person=20, resize=0.5, color=True)
18 print(f"Images chargees : {len(lfw.images)}")
19
20 # Cellule 4: Calcul embeddings (voir implementation complete dans le chapitre 4)
```

Listing A.1 – notebook.ipynb (extrait principal)

A.2 Application Streamlit

Le code complet de `app.py` est disponible dans le dossier projet et fait environ 250 lignes. Les extraits principaux ont été présentés dans le chapitre 5.

Annexe B

Captures d'écran de l'interface

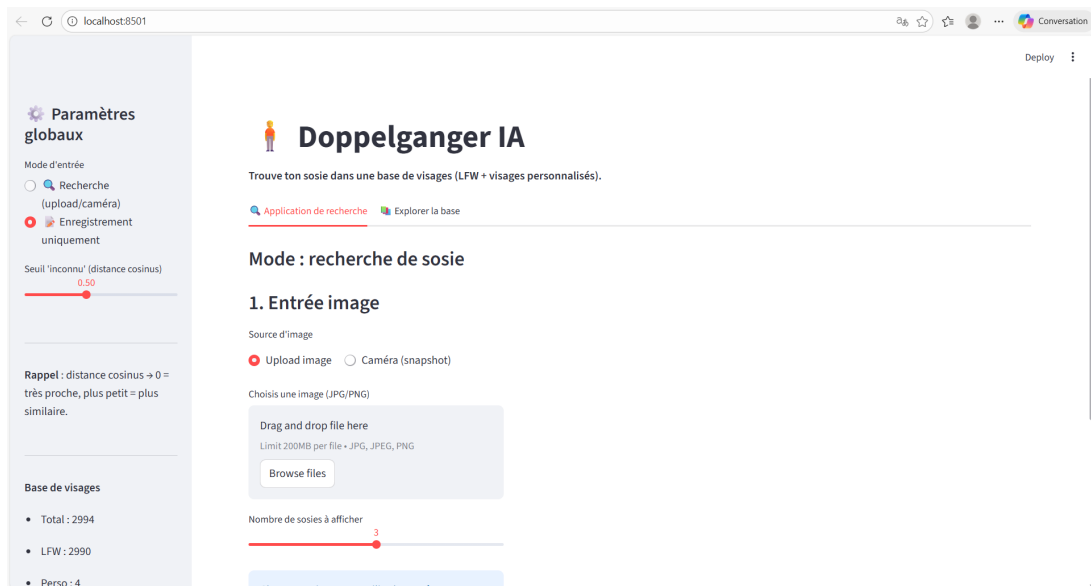


FIGURE B.1 – Interface mode Upload avec résultats de recherche

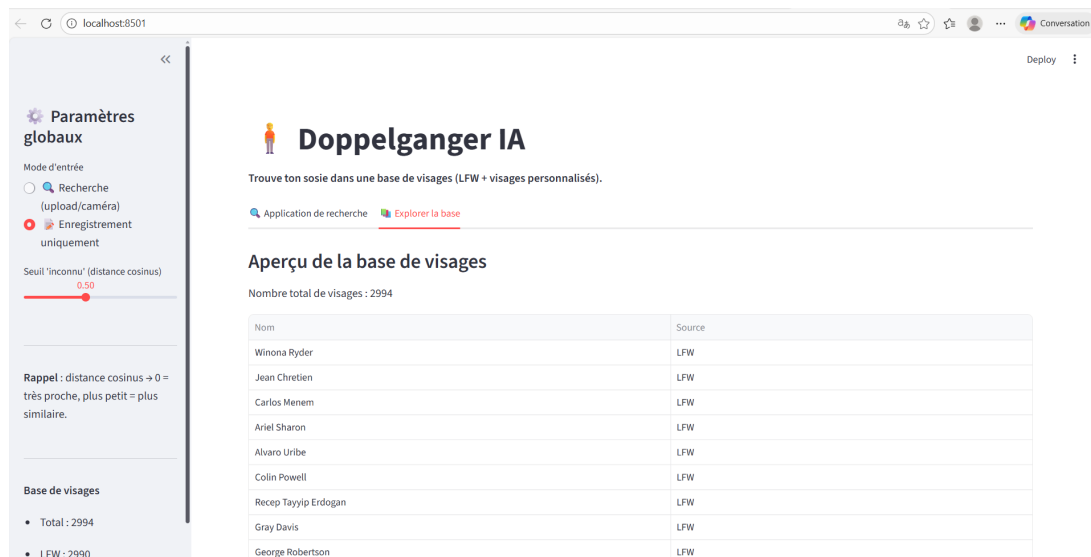


FIGURE B.2 – Onglet Explorer la base avec affichage des visages

Annexe C

Résultats détaillés

C.1 Exemples de sosies trouvés

Query	Top-3 sosies	Distances
George W. Bush	Bush, Bush, Colin Powell	0.18, 0.23, 0.52
Tony Blair	Blair, Blair, Jacques Chirac	0.21, 0.25, 0.48
Utilisateur test 1	Hugo Chavez, Tony Blair, Ariel Sharon	0.68, 0.71, 0.74

TABLE C.1 – Exemples de résultats de recherche

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Mes remerciements s'adressent tout particulièrement à :

- Notre encadreur **Mohamed Abbas Konate**, pour ses conseils avisés, son soutien et son accompagnement tout au long du projet